

에이전트 루프·스트리밍·도구 런타임

한 사용자 턴이 모델과 환경을 왕복하며 수렴하는 과정

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

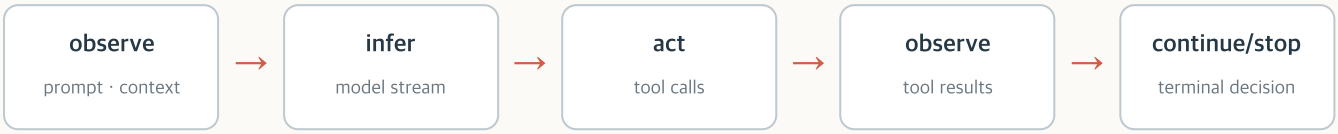
독자
agent runtime 구현자

사용 방식
상태기계 · 실패 경로 분석

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

에이전트는 모델이 아니라 폐루프 시스템이다

모델 출력이 tool action을 만들고, 환경 결과가 다시 모델 입력이 되는 feedback loop가 핵심이다.



ReAct 연구는 reasoning과 action을 교차시키는 패턴을 제시했다. 실제 제품 harness는 여기에 권한·취소·압축·재시도·관측·UI를 더한다.

근거 위치 ReAct paper · Toolformer paper · src/query.ts

query와 queryLoop의 역할 분리

public generator는 event stream과 terminal을 노출하고, 내부 loop는 반복 상태와 회복 정책을 소유한다.

함수	소유 책임	중요 semantics
`query()`	public async generator facade	yield delegation, 정상 종료 후 command lifecycle 완료
`queryLoop()`	turn iteration과 상태 전이	while(true), continue/return/throw
`queryModelWithStreaming()`	model adapter facade	VCR wrapper와 하위 stream 위임
`runTools()`	tool batch orchestration	safe partition, serial/concurrent merge
`runToolUse()`	단일 capability 실행	lookup, parse, permission, hook, call, normalize

근거 위치 src/query.ts · src/services/api/claude.ts · src/services/tools/toolOrchestration.ts

iteration을 건너가는 최소 상태

loop-local state는 메시지, tool context, 압축 tracking, recovery count, turn count, 이전 transition을 묶는다.

필드	왜 필요한가	잘못 다루면
messages	다음 model request의 history	tool_use/result 불일치
toolUseContext	도구·권한·앱 상태·취소	stale capability/permission
autoCompactTracking	압축 이후 turn·실패 추적	무한 재압축
recovery count	bounded retry	비용 폭증·livelock
pending summary	tool 요약을 다음 요청과 겹쳐 실행	불필요한 latency
transition	continue 이유를 테스트/관측	회복 경로가 보이지 않음

근거 위치 src/query.ts

한 번의 while(true) 해부

각 iteration은 “입력 projection → request → stream → tool → enrichment → next state”의 transaction-like 경계를 만든다.

1. 현재 State를 iteration-local 변수로 펼친다.
2. memory/skill prefetch를 시작해 model/tool 시간과 겹친다.
3. snip·microcompact·context collapse·autocompact로 messages projection을 만든다.
4. token blocking 조건을 확인한다.
5. model stream을 소비하며 event를 yield하고 tool_use를 수집한다.
6. recoverable error면 표시를 보류하고 recovery branch를 평가한다.
7. tool이 없으면 stop hook/budget을 확인하고 terminal return한다.
8. tool batch를 실행하고 result/attachment/context update를 모은다.
9. max turn을 확인하고 next State를 만든 뒤 continue한다.

근거 위치 src/query.ts · src/query.ts · src/query.ts

컨텍스트는 하나의 배열이 아니라 여러 projection의 합성

긴 대화의 유한 context window를 관리하기 위해 서로 다른 비용·정확도 특성을 가진 압축 단계가 순서대로 적용된다.

단계	의도	핵심 주의
tool result budget	과대 결과를 저장/축약	Read 같은 도구는 순환을 막기 위해 예외
snip	선택적 과거 제거	token freed를 후속 threshold에 반영
microcompact	tool-use id 기반 작은 정리	API cache deletion과 합성
context collapse	세분화된 projection/commit	feature gated, full history와 구분
autocompact	요약 메시지로 큰 경계 생성	실패 circuit breaker, task budget

> **중요한 구조**
>
> 압축은 iteration 시작의 전처리가 아니라, 실패 회복·budget·세션 resume·tool result persistence와 얽힌 상태 전이다.

근거 위치 src/query.ts · src/services/compact/compact.ts

모델 호출은 많은 정책의 합류점

history 외에도 system/user context, tools, model, thinking, effort, MCP, fallback, budget, signal이 request에 결합된다.

요청 모양의 개념 축약

```
callModel({
  messages: project(history, userContext),
  systemPrompt: append(systemPrompt, systemContext),
  tools,
  signal,
  options: { model, fallbackModel, mcpTools, effort, taskBudget }
})
```

캐시 불변식

observable input을 보강할 때 API로 되돌아갈 원본을 mutate하지 않는 주석이 있다. byte-level prompt cache identity가 기능 정확도와 비용에 영향을 준다.

근거 위치 src/query.ts · src/query.ts

이벤트는 즉시 내보내고, 일부는 내부에도 축적한다

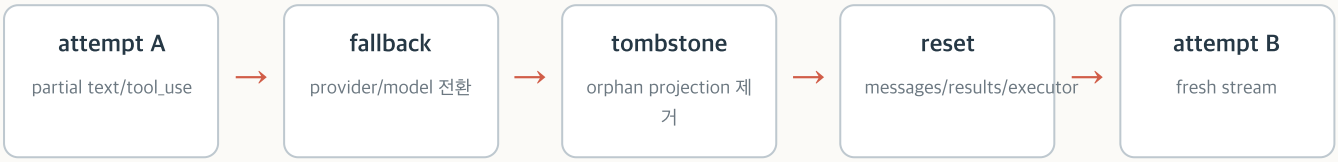
같은 message가 UI/SDK로 yield되는 동시에 assistantMessages 배열에 쌓여 tool discovery와 다음 request 구성에 쓰인다.

stream 처리	외부 projection	내부 상태
일반 text/thinking event	즉시 yield	assistantMessages에 축적
tool_use block	message yield	toolUseBlocks + needsFollowUp
streaming tool result	완료 즉시 yield	normalized toolResults
recoverable error	일시 withhold	recovery 판단용 축적
fallback orphan	tombstone yield	이전 partial arrays 초기화

근거 위치 src/query.ts

fallback에서 partial output을 지우는 이유

streaming attempt가 도중 fallback으로 바뀌면 이미 노출된 partial assistant message가 새 응답과 공존해 transcript·signature·tool id를 오염시킬 수 있다.



일반화

streaming UI에는 append만이 아니라 retract/replace semantics가 필요하다. event log와 materialized view를 분리하면 이런 회복을 표현하기 쉽다.

근거 위치 src/query.ts

stop_reason만 믿지 않는다

소스 주석은 API의 `stop_reason === tool_use`가 항상 신뢰할 수 없다고 밝히고, 실제 stream에서 tool_use block을 관찰한 사실을 loop continuation 신호로 쓴다.

이 설계는 “summary metadata”보다 “실제 payload evidence”를 우선한다. 분산·스트리밍 시스템에서 상태 플래그보다 관찰된 event를 기준으로 전이하는 전형적인 방어다.

> **검증 질문**
> 중복 tool_use, partial JSON, fallback 후 old tool id, unknown tool alias가 각각 어떤 오류 메시지와 cleanup을 만드는가?

근거 위치 src/query.ts · src/query.ts

tool_use에서 외부 효과까지

모델의 JSON 제안은 곧바로 실행되지 않는다. lookup·schema·validation·permission·hook·call·result mapping 단계를 통과한다.



unknown tool은 예외만 던지는 대신 `tool_result` error로 모델에게 되돌려 self-correction을 가능하게 한다.

근거 위치 src/services/tools/toolExecution.ts · src/Tool.ts

동시성은 입력별 capability 속성

같은 Tool이라도 input에 따라 concurrency-safe 여부가 달라질 수 있다. 호출 배열을 안전/비안전 블록으로 partition한다.

경로	실행 방식	context update
safe block	bounded concurrent generator merge	modifier를 toolUseID별로 큐잉 후 결정적 적용
unsafe block	한 호출씩 serial	각 결과 직후 currentContext 갱신
parse/판정 실패	보수적으로 unsafe	직렬화
기본값	`isConcurrencySafe=false`	명시적 opt-in

> **기본 동시성 10**
> 환경 변수로 상한을 바꿀 수 있지만, resource별 rate limit·FD·subprocess 비용을 자동으로 이해하는 값은 아니다. 부하 시험이 필요하다.

근거 위치 src/services/tools/toolOrchestration.ts

정책 결정과 lifecycle 확장은 도구 실행 안에 있다

PreToolUse·PermissionRequest·PostToolUse·PostToolUseFailure가 tool call 경계를 감싼다.

지점	가능한 결과	권위
input validation	유효/오류	tool contract
pre-tool hook	allow/deny/update/defer	설정된 hook 정책
permission resolver	allow/deny/ask	mode·rule·user decision
tool call	output/progress/throw	capability adapter
post hook	annotation/context/failure handling	hook contract

근거 위치 src/services/tools/toolExecution.ts · 공식 hooks reference

tool result가 다음 모델 턴을 만든다

tool execution이 끝나면 assistant message와 tool result, 새 attachment를 순서 규칙에 맞춰 합치고 next State를 만든다.

다음 iteration 전이의 개념 축약

```
next.messages = [
  ...messagesForQuery,
  ...assistantMessages,
  ...toolResults
]
next.turnCount = turnCount + 1
next.transition = { reason: "next_turn" }
```

API는 tool_use 뒤 대응하는 tool_result의 순서와 id를 요구한다. queue command나 file-change attachment는 tool batch 뒤에 붙어야 한다는 주석이 이를 지킨다.

근거 위치 src/query.ts · src/query.ts

종료는 boolean이 아니라 이유가 있는 값

completed, blocking limit, stop-hook prevented, aborted tools, hook stopped, max turns 등은 caller가 서로 다르게 처리해야 한다.

종료 이유	성격	후속 질문
completed	정상 수렴	completion hooks/usage 정리됐는가?
blocking_limit	컨텍스트 한계	manual compact 안내?
stop_hook_prevented	정책 중단	사용자에게 이유가 보이는가?
aborted_tools	취소	부분 효과와 subprocess가 정리됐는가?
hook_stopped	hook이 continuation 차단	result가 모델에 전달됐는가?
max_turns	운영 한도	현재 상태를 재개할 수 있는가?

근거 위치 Terminal/Continue import 지점 · src/query.ts · src/query.ts · src/query.ts

> **근거 수준**
>
> `query/transitions.ts` 자체는 복원본에서 누락됐다. 종료 variant는 호출·반환 지점으로 교차 확인해야 한다.

오류를 즉시 보여 주지 않는 경우

prompt-too-long, media size, max-output-tokens처럼 자동 회복 가능한 오류는 UI/SDK에 먼저 노출하면 consumer가 세션을 끝낼 수 있어 일시 보류한다.



회복 불변식

withhold 조건과 recovery gate는 같은 snapshot을 사용해야 한다. gate가 stream 중 바뀌면 “숨겼지만 회복하지 않는” 이벤트 손실이 생길 수 있다는 주석이 이를 설명한다.

근거 위치 src/query.ts · src/query.ts

복잡한 loop는 checkpoint와 correlation 없이는 보이지 않는다

queryChainId/depth, requestId, toolUseID, 단계 checkpoint, transition reason을 통해 한 turn의 causal path를 조립한다.

식별자/신호	결합 대상	진단 질문
queryChainId + depth	main/subagent request chain	어느 재귀/후속 턴인가?
requestId	provider request·fallback	어느 API 시도가 실패했나?
toolUseID	tool_use ↔ result/progress	orphan result인가?
checkpoint	compaction/API/tool phases	latency가 어디서 생겼나?
transition.reason	continue path	왜 loop가 한 번 더 돌았나?

근거 위치 src/query.ts · src/utils/queryProfiler.ts

코드에서 추출한 핵심 불변식

함수 흐름보다 불변식을 적어야 테스트와 재구현에 사용할 수 있다.

- ☐ 모든 tool_use id에는 정확히 대응하는 tool_result 또는 명시적 interruption이 있어야 한다.
- ☐ fallback 전 partial assistant projection은 새 stream과 공존하지 않는다.
- ☐ API로 재전송할 original message는 observer용 backfill 때문에 mutate되지 않는다.
- ☐ permission/validation을 통과하지 않은 tool effect는 실행되지 않는다.
- ☐ concurrency-unsafe tool의 context modifier는 직렬 순서로 적용된다.
- ☐ recoverable error는 recovery가 불가능하다고 결정되기 전 외부 terminal로 누출되지 않는다.
- ☐ max-turn·token-budget·recovery retry에는 상한이 있다.
- ☐ abort는 rollback이 아니므로 부분 효과를 별도로 정리한다.

관찰 가능한 축약 루프 만들기

행복 경로보다 failure matrix를 먼저 만든다.

실험	주입할 사건	기대 evidence
text only	text → stop	yield 순서 + completed
one tool	tool_use → result → text	두 model request + id pairing
unknown tool	없는 name	is_error result + self-correction 가능
fallback	partial text 후 provider error	tombstone + fresh stream
abort	tool 50% 시점 signal	cleanup + aborted_tools
context limit	oversized history	compact 또는 blocking_limit
hook block	PreToolUse deny	effect 없음 + reason

실습 · event ledger

각 실험에서 시간, event type, requestId, toolUseID, state transition을 NDJSON으로 기록한다.

완료 증거: 정렬 가능한 7개 trace와 invariant 검사

루프를 이해했다는 기준

while문을 찾는 것이 아니라 어떤 evidence가 어떤 상태 전이를 정당화하는지 설명할 수 있어야 한다.

CONTROL

Generator

stream event와 terminal을 하나의 protocol로 묶는다.

STATE

Explicit transition

회복·budget·turn의 이유를 데이터로 남긴다.

EFFECT

Tool pipeline

모델 제안을 capability effect로 안전하게 번역한다.

RECOVERY

Projection repair

tombstone·withhold·compact로 일관된 관찰을 유지한다.

다음 권

04권은 이 event stream을 사용자가 보는 terminal frame으로 바꾸는 React/Ink 경계를 해부한다.